Práctica 1 — Ficha de resultados

Inteligencia Artificial Avanzada (IA4) · Curso 2026/2027

> Todos los números salen de **referencia_s42.csv** (semilla 42, 40 partidas por punto), salvo las pruebas de humo, que salen de los modos `traza` y `duelo`, y la fila de calibración de E4, que sale de `calibrar 42` (`calibracion_s42.csv`).

Datos

Campo	Valor
Alumno/a 1	Pol Gómez Morales
Alumno/a 2	Daniel Cabezas Cappa
Semilla usada	`42`
Partidas por punto	`40`
Fichero CSV entregado	`referencia_s42.csv`
¿Se ha tocado algún fichero además de MotorMcts.cs?	No. Sólo `MotorMcts.cs`.
Procesador y sistema (E4 depende de la máquina)	Intel Core i9-14900K · 32 GB DDR5 · Windows 11

Pruebas de humo

Prueba	Resultado esperado	Lo que sale
traza elige la columna 3 en los seis presupuestos	sí	`sí (6 de 6)`
% de visitas de la columna 3 con 25 simulaciones	≈ 28 %	`28,0 % (7/25)`
% de visitas de la columna 3 con 10.000 simulaciones	≈ 95 %	`94,7 % (9.466/10.000)`
duelo 40 2000 4 42	> 45 %	`56,3 % (22-17-1)`
MCTS-500 contra Aleatorio	≈ 100 %	`100 % (40-0-0)`

E1 · presupuesto de simulaciones

Iteraciones	vs Aleatorio (%)	vs Minimax-4 (%)	ms por jugada (vs Aleatorio · vs Minimax-4)
50	`97,5`	`2,5`	`0,3 · 0,3`
100	`100,0`	`2,5`	`0,5 · 0,5`
200	`100,0`	`15,0`	`1,0 · 1,0`
500	`100,0`	`40,0`	`2,7 · 2,1`
1.000	`100,0`	`48,8`	`5,5 · 4,4`
2.000	`100,0`	`56,3`	`10,6 · 8,0`
5.000	`100,0`	`61,3`	`27,4 · 21,3`

¿A partir de cuántas simulaciones deja de perder contra Minimax-4?

Con 1.000 iteraciones llega al empate técnico (48,8 %, 19-20-1) y desde 2.000 gana más de lo que pierde (56,3 %, 22-17-1). Con 500 todavía pierde (40,0 %).

E2 · con heurística y sin ella · MCTS-2.000

Profundidad del rival	Minimax con heurística (%)	Minimax ciego (%)	Diferencia (puntos)
2	58,8	96,3	37,5
4	56,3	92,5	36,2
6	23,8	92,5	68,7

¿Cuánto mejora el rival ciego al pasar de profundidad 4 a 6?

Nada: MCTS saca 92,5 % en ambos casos (36-2-2). Mejora 0 puntos.

¿Y el rival con heurística?

Mucho: MCTS pasa de 56,3 % a 23,8 %, así que el rival mejora 32,5 puntos.

E3 · la constante de exploración

C	0	0,5	1	1,41	2	4
% contra Minimax-4	5,0	70,0	57,5	56,3	61,3	40,0

¿Qué le pasa a MCTS con C = 0? ¿Y con C = 4? Un nombre para cada fallo:

C = 0 → explotación pura (avaricia, convergencia prematura): tras probar cada hijo una vez, se queda con el que tuvo suerte en su primer rollout y no vuelve a mirar los demás. Resultado: 5,0 %.
C = 4 → sobreexploración: reparte las visitas casi por igual entre jugadas buenas y malas, el árbol no profundiza y se comporta casi como un Monte Carlo plano. Resultado: 40,0 %.

E4 · presupuesto de tiempo

ms por jugada	5	10	25	50	100
Iteraciones que caben (modo calibrar)	621	1.505	3.410	5.180	12.395
% contra Minimax-4	55,0	52,5	61,3	72,5	65,0

La calibración incluida en referencia_s42.csv (filas E4cal) da 805 · 1.339 · 3.506 · 5.933 · 12.695. Las dos son medidas de reloj y por eso no coinciden exactamente; la mayor diferencia está en 5 ms (621 frente a 805), donde la medida es más sensible al ruido.

Con 16 ms —un frame a 60 fps—, ¿cuántas iteraciones caben en vuestra máquina? Mirando E1, ¿qué porcentaje da ese presupuesto? ¿Lo pondríais en un juego?

Unas **2.300 iteraciones** (interpolando entre 1.505 en 10 ms y 3.410 en 25 ms). En E1, 2.000 iteraciones dan **56,3 % contra Minimax-4** y 100 % contra el aleatorio; en E4, 16 ms queda entre el 52,5 % y el 61,3 %. **Sí, lo pondríamos**: ya gana al alfa-beta-4 con heurística y es *anytime*. Pero no le daríamos el frame entero, que también necesitan el render y la lógica: repartiríamos la búsqueda en varios frames o la lanzaríamos en un hilo aparte.

Opcionales realizadas

Opción	Hecha	Qué salió
O1 reutilizar el árbol	No	—
O2 rollout con política	No	—
O3 Monte Carlo plano	No	—
O4 gráficas	Sí	Gráficas de E1, E3 y E4 generadas desde referencia_s42.csv (Python + matplotlib, `graficas.py`), incluidas en el informe. E1 sale creciente y E3 hundido en los dos extremos, como se esperaba. E4 tiende a crecer pero no es monótona (10 ms y 100 ms caen 2,5 y 7,5 puntos), dentro del ruido de ±8 de 40 partidas.

Conclusión

¿Cuándo usaríais MCTS y cuándo alfa-beta? (cuatro líneas)

MCTS cuando no hay una buena función de evaluación, la ramificación es alta o hace falta una respuesta *anytime* con presupuesto variable: sin heurística, el minimax no mejora con la profundidad y MCTS le gana con un 92,5 %. Alfa-beta cuando hay una heurística fiable y el juego es táctico, de ramificación baja y con trampas cortas que los rollouts aleatorios no ven: con heurística y profundidad 6 deja a MCTS-2000 en un 23,8 %. El alfa-beta vale lo que vale su heurística; MCTS, lo que valen sus simulaciones.